SIMATS ENGINEERING

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

COMPUTER VISION | ITA0515 | SLOT B

SOLVED ANSWERS

.

1. Real-Time Face Recognition System (Edge Deployment)

Problem: A face-recognition system must run on a low-power edge device such as a Raspberry Pi, process live video in real time, tolerate varying lighting, use limited memory and CPU, and recognize faces from a predefined dataset.

A. Constraints Identified

- Real-time processing: the design must keep processing delay low and target at least about 30 FPS where the hardware permits.
- Varying illumination: preprocessing must reduce the effect of brightness and contrast changes.
- Limited CPU and memory: lightweight algorithms, reduced frame size and controlled processing frequency are required.
- Predefined identities: the recognition stage should compare detected faces with a stored gallery of known persons.

B. Proposed OpenCV Pipeline

Stage	OpenCV Approach	Purpose
1. Image acquisition	cv2.VideoCapture(0)	Capture frames from the live camera.
2. Frame reduction	cv2.resize()	Reduce resolution to lower CPU and memory usage.
3. Preprocessing	cv2.cvtColor(), cv2.equalizeHist() / CLAHE	Convert to grayscale and improve illumination robustness.
4. Face detection	Haar Cascade / lightweight DNN detector	Locate face regions in each frame.
5. Feature extraction	LBPH-based face representation	Represent each detected face using local texture information.
6. Recognition	LBPHFaceRecognizer	Compare the detected face with the predefined training dataset.
7. Output	cv2.rectangle(), cv2.putText()	Draw the face box, identity and confidence on the frame.

C. Design Procedure

- Collect a small predefined dataset containing multiple images for each authorized person.
- Convert training images to grayscale and detect/crop the face region.
- Train an LBPH recognizer using the cropped face images and identity labels.
- Open the camera using cv2.VideoCapture().
- Resize incoming frames before detection to reduce computation.
- Apply grayscale conversion and illumination normalization.
- Run face detection. Only the detected face regions are passed to recognition.
- Recognize each detected face using the trained LBPH model.
- Display the recognized identity and confidence together with the bounding rectangle.

- Release the camera and destroy windows when the application terminates.

D. Constraint Analysis and Justification

Constraint	Design Decision	Why It Satisfies the Constraint
Real-time	Resize frames; process at controlled resolution; use lightweight detection.	Reduces per-frame computation and latency.
Lighting changes	Grayscale + histogram/CLAHE enhancement.	Improves local contrast and reduces illumination sensitivity.
Low resources	LBPH, small frames, limited detection frequency.	Avoids a heavy recognition pipeline and reduces CPU/RAM demand.
Predefined dataset	Train recognizer with known identities.	Allows direct comparison against the stored identity gallery.

2. Automated Video Surveillance with Alert System

Problem: The surveillance system must detect motion and unusual activities, generate real-time alerts, accept standard CCTV feeds and minimize false positives in dynamic environments.

A. Proposed End-to-End Pipeline

Stage	OpenCV Technique	Function
1. CCTV input	cv2.VideoCapture()	Read live CCTV/video frames.
2. Preprocessing	resize(), cvtColor(), GaussianBlur()	Normalize size, convert to grayscale and suppress noise.
3. Background model	Background subtraction such as MOG2/KNN	Separate moving foreground from the background.
4. Morphological cleanup	cv2.morphologyEx()	Remove small noise and connect fragmented regions.
5. Motion detection	Contours / connected components	Find moving objects and their bounding boxes.
6. Activity analysis	Area, position, duration and movement rules	Reject small noise and identify suspicious movement.
7. Alert generation	Threshold + event logic	Trigger an alert only when conditions persist.
8. Visualization	rectangle(), putText()	Show detected regions and alert status.

B. Handling Noise and Dynamic Backgrounds

- Apply Gaussian blur before motion analysis to reduce sensor noise and small intensity variations.
- Use adaptive background subtraction such as MOG2 or KNN rather than a single fixed background image.
- Apply opening to remove isolated foreground noise and closing to fill small gaps in detected objects.
- Use a minimum contour-area threshold so tiny moving pixels do not create alerts.
- Require motion to persist for several consecutive frames before classifying it as an event.
- Use a region of interest (ROI) to monitor only relevant areas such as entrances, restricted zones or roads.

C. Reducing False Positives

- Ignore foreground blobs smaller than a predefined area threshold.
- Use temporal confirmation: an alert is generated only if the object remains active for multiple frames.
- Use location rules so movement outside the monitored ROI is ignored.
- Optionally combine motion with object/feature detection to distinguish meaningful objects from shadows and minor background changes.
- Add an alert cooldown period to prevent repeated alerts for the same event.

D. Real-Time Considerations

- Resize frames to a processing resolution appropriate for the CCTV hardware.
- Use efficient OpenCV operations and avoid unnecessary copies of frames.
- Process only the required ROI when full-frame analysis is unnecessary.
- Keep alert generation lightweight and asynchronous where possible.

3. OCR System for Low-Quality Documents

Problem: The OCR system must extract text from low-resolution, noisy scanned documents, handle varying fonts and illumination, and process documents efficiently in batches.

A. Proposed Image Processing Pipeline

Stage	OpenCV Operation	Purpose
1. Image input	cv2.imread()	Load each scanned document.
2. Grayscale	cv2.cvtColor()	Remove color information and simplify processing.
3. Denoising	cv2.GaussianBlur() / medianBlur()	Reduce scan noise while preserving text structure.
4. Contrast enhancement	CLAHE / histogram equalization	Improve text-background separation under uneven illumination.
5. Thresholding	cv2.threshold() / adaptiveThreshold()	Convert the document into a suitable binary representation.
6. Morphology	cv2.morphologyEx()	Remove speckles and repair broken character strokes.
7. Segmentation	Contours / connected components	Identify text regions or lines.
8. Deskew/correction	Rotation based on text orientation	Correct slanted scans before OCR.
9. OCR	Pass processed regions to OCR engine	Recognize the cleaned text.
10. Output	Text file/database	Store extracted text for later use.

B. Why Each Stage Is Necessary

- Grayscale reduces the image to one intensity channel, lowering processing cost.
- Denoising removes isolated scan artifacts that could be mistaken for character strokes.
- CLAHE is useful when illumination varies across the page because it enhances local contrast.
- Adaptive thresholding is preferable when the background is uneven because it uses local intensity information.
- Morphological opening removes small noise, while closing can reconnect small gaps in characters.
- Deskewing aligns text lines, improving segmentation and OCR recognition.
- Segmentation isolates text regions so that the OCR engine receives cleaner input.

C. Batch Processing Strategy

- Read documents one by one from the input folder.
- Apply the same preprocessing pipeline to every page.
- Store intermediate results only when required to avoid unnecessary memory use.
- Process each document independently and save the extracted text immediately.
- Use consistent image dimensions and preprocessing parameters across the batch.

4. Facial Expression Recognition for Mobile Application

Problem: The mobile application must identify happy, sad and neutral expressions while operating on limited mobile CPU resources, maintaining real-time performance, and tolerating slight head movement and lighting changes.

A. Proposed Pipeline

Stage	OpenCV Approach	Purpose
1. Camera input	cv2.VideoCapture() / mobile camera interface	Capture live face images.
2. Frame resizing	cv2.resize()	Reduce computation while retaining facial information.
3. Illumination handling	Grayscale + CLAHE	Improve robustness to moderate lighting variation.
4. Face detection	Haar Cascade / lightweight detector	Locate the face before expression analysis.
5. Face alignment	Eye/landmark-based alignment	Reduce effects of slight head movement.
6. Expression features	Normalized facial ROI / lightweight feature representation	Extract information related to expression.
7. Classification	Three-class classifier: happy, sad, neutral	Assign the expression label.
8. Display	rectangle() + putText()	Show face box and expression label.

B. Balancing Accuracy and Efficiency

- Use a small input resolution for face detection and recognition.
- Detect the face periodically and track the detected region between detections where appropriate.
- Normalize the face crop to a fixed size before classification.
- Use grayscale and local contrast enhancement instead of expensive full-color processing.
- Restrict the classifier to the three required classes: happy, sad and neutral.
- Use temporal smoothing, such as majority voting across recent frames, to prevent rapid label changes.

C. Handling Head Movement and Lighting

- Detect the face with a bounding box and crop the face region before classification.
- Use eye positions or other facial reference points to align the face when slight rotation occurs.
- Apply CLAHE or suitable normalization to reduce moderate illumination changes.
- Use a confidence threshold and retain the previous stable expression when the current frame is uncertain.

5. Smart Traffic Monitoring System

Problem: The city authority requires live vehicle detection, tracking across frames, vehicle counting, statistics display and continuous operation with minimal delay.

A. Complete OpenCV Pipeline

Stage	OpenCV Method	Purpose
1. Video input	cv2.VideoCapture()	Read the live traffic camera stream.
2. Preprocessing	resize(), blur(), ROI masking	Reduce computation and focus on the roadway.
3. Vehicle detection	Background subtraction / contour detection or detector	Locate vehicles in each frame.
4. Bounding boxes	cv2.boundingRect()	Represent detected vehicles spatially.
5. Tracking	Centroid/IoU-based association	Match detections between consecutive frames.
6. Counting	Virtual line / region crossing	Increment vehicle count when tracked objects cross the counting line.
7. Statistics	Counters and frame-time calculations	Maintain vehicle totals and other displayed metrics.
8. Visualization	rectangle(), line(), putText()	Display boxes, tracks, counting line and statistics.

B. Vehicle Detection and Tracking

- Capture frames continuously from the traffic camera.
- Resize frames to a practical processing resolution.
- Define a road ROI so irrelevant areas are excluded.
- Use background subtraction or another suitable detector to identify moving vehicles.
- Apply blur and morphological operations to reduce noise and connect fragmented foreground regions.
- Find contours and discard objects below a minimum vehicle-area threshold.
- Calculate each vehicle's centroid from its bounding box.
- Associate current centroids with previous-frame centroids using distance or another simple tracking rule.
- Assign a persistent ID to each tracked vehicle so the same vehicle is not counted repeatedly.

C. Vehicle Counting and Statistics

- Draw a virtual counting line across the road using cv2.line().
- Store the previous and current centroid position for each tracked vehicle.
- If a vehicle centroid crosses the line in the required direction, increment the counter once.
- Use a set of counted vehicle IDs to prevent double counting.
- Display the total count with cv2.putText().
- Optionally display processing FPS, current number of tracked vehicles and directional counts.

D. Ensuring Accuracy and Real-Time Performance

Requirement	Design Choice	Effect
Live detection	ROI + resized frames	Reduces unnecessary processing.
Tracking	Persistent IDs	Prevents repeated counting of the same vehicle.
Noise handling	Blur + morphology + area threshold	Reduces false detections.
Continuous operation	Efficient frame loop and resource release	Supports long-running monitoring.
Low delay	Lightweight operations and limited resolution	Improves real-time responsiveness.
Visualization	OpenCV drawing functions	Provides immediate traffic statistics.